

用户手册

概览

Tier0 是什么？

什么是 Tier0？

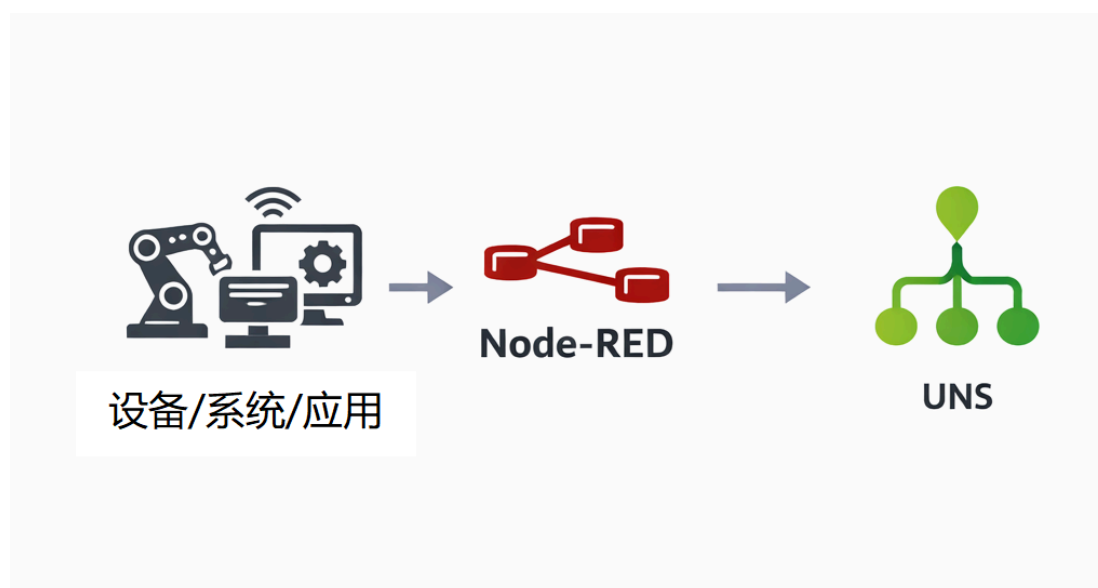
Tier0 是一个基于统一命名空间（UNS）的工业数据平台。

它通过 Node-RED 连接数据，将其组织为统一结构，并提供给上层应用和业务分析使用。

它如何工作？

Tier0 在一个统一的流程中对数据建模，处理数据：

- Node-RED 从设备和系统中连接并处理数据
- 数据被发布到统一命名空间（UNS），并在 Tier0 数据库中结构化存储
- 数据被仪表板、应用程序和分析工具消费



核心组件

统一命名空间（UNS）

为所有工业数据定义分层结构。

factory/workshopA/line1/cnc01/telemetry/spindleSpeed

工厂模型

工厂模型 模板 标签

请输入名称/别名/路径进行搜

Metric(1)

factory(1)

workshopA(1)

line1(1)

cnc01(1)

telemetry(1)

spindleSpeed

factory / workshopA / line1 / cnc01 / telemetry / Metric / spindleSpeed

导入 导出

spindleSpeed

明细

定义

消息体

数据看板

键	类型	长度	显示名称	备注
timeStamp	DATE TIME			
speed	INTEGER			
quality	LONG			

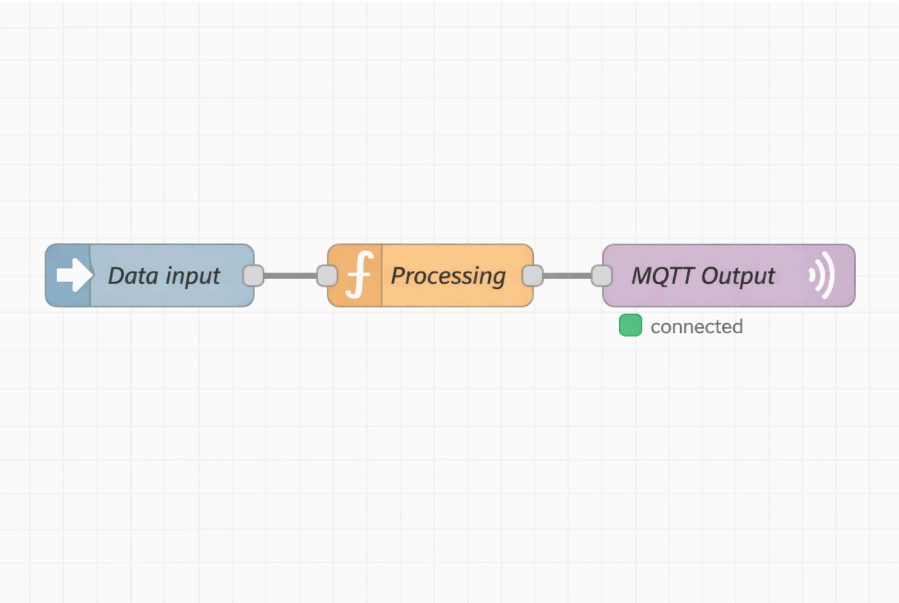
键	值	更新时间
---	---	------

创建

Node-RED 集成

用于数据连接与处理的核心引擎。

- 连接设备、应用和系统
- 将数据格式化并发布到命名空间中



功能列表

本节列出了 Tier0 的功能，每个功能在文档中都有对应的使用指南。您可以点击链接了解如何使用各项功能。

功能	说明	备注
数据建模	将工业数据建模为层级结构（文件夹-文件），以提升清晰度。	- 私有化部署。 - 有关规格详情，请参见 系统要求。
数据连接	通过 Node-RED 以拖拽方式连接多种数据源。	

入门指导

部署 Tier0

系统要求

配置类型	CPU	内存	硬盘/IOPS
基础配置	4核	8GB	100GB, 2000 IOPS (30%写入)
推荐配置	8核	16GB	1TB, 2000 IOPS (30%写入)

安装 Tier0

Linux 系统

1. 确保准备好以下系统和组件：

- Ubuntu Server 24.04
- Docker

组件	版本
Docker Engine - Community	27.4.0
Docker Buildx	v0.19.2
Docker Compose	v2.31.0
Containerd	1.7.24

① 备注

上述系统和组件均已测试通过，您也可以尝试其他版本。

2. 从 Github 克隆项目：

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

3. 进入项目内的 Tier0-Edge/deploy 文件夹:

```
cd Tier0-Edge/deploy
```

4. 打开 .env.default 文件, 按 i 键根据需要编辑配置项:

```
vi .env.default
```

配置项	说明
VOLUMES_PATH	项目数据的存储路径。
ENTRANCE_DOMAIN	Tier0 的访问地址, 通常是安装 Tier0 的服务器 IP 或域名。 ⚠ 注意 请勿使用 127.0.0.1 或 localhost, 否则登录和认证功能将无法正常运行。
LANGUAGE	Tier0 的显示语言, 支持中文 (zh-CN) 和英文 (en-US)。

```
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=linux

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=1
OS_NAME=Tier0
VOLUMES_PATH=/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work)
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192.168.1.100
ENTRANCE_PORT=8088
ENTRANCE_SSL_PORT=8443

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US
```

5. 按 ESC 停止编辑, 保存文件并安装 Tier0:

```
:wq!
bash bin/install.sh
```

6. 选择是否保留输入的访问地址、是否使用默认设置, 然后等待安装完成。

```

Available options for ENTRANCE_DOMAIN:
0). Keep current: office.unibutton.com (default)
1). 192.168.31.167
2). 172.17.0.1
3). 172.18.0.1
Select option (0-3), or press Enter to keep current: 0
Keeping current ENTRANCE_DOMAIN: office.unibutton.com
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /root/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map

```

7. 在浏览器中访问 Tier0 并登录。



Windows 系统

① 信息

- 确保已在 Windows 10 或 11 上安装最新版 Docker Desktop 和 Git。
- 启动 Docker。

1. 在 Git Bash 中从 Github 克隆项目：

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

2. 进入项目内的 Tier0-Edge/deploy 文件夹:

```
cd Tier0-Edge/deploy
```

3. 编辑 .env.default 文件中的以下属性:

✓ 提示

编辑 .env.default 文件前, 在 Git Bash 中执行 `ipconfig` 获取服务器 IP。

```
C:\Users\support>ipconfig /all

Windows IP Configuration

Host Name . . . . . : -10x64-T
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet0:

Connection-specific DNS Suffix . :
Description . . . . . : Ethernet Adapter
Physical Address. . . . . : 00-5
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80:: (Preferred)
IPv4 Address. . . . . : 192.168. (Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Friday, June 9, 2023 1:47:47 AM
Lease Expires . . . . . : Friday, June 9, 2023 5:07:47 AM
Default Gateway . . . . . : 192.168.
DHCP Server . . . . . : 192.168.
DHCPv6 IAID . . . . . : 503
DHCPv6 Client DUID. . . . . : 00-01-00-
DNS Servers . . . . . : 192.168.
NetBIOS over Tcpip. . . . . : Disabled
```

- OS_PLATFORM_TYPE: 安装 Tier0 的操作系统, 改为 windows
- VOLUMES_PATH: 项目数据的存储路径。
- ENTRANCE_DOMAIN/ENTRANCE_PORT: Tier0 的访问地址, 通常是安装 Tier0 的服务器 IP 或域名, 确保端口未被占用。

```
vi .env.default
```

① 备注

.env.default 文件默认隐藏, 直接输入命令即可打开。

```

MINGW64/d/supOS-CE
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=windows

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=2
OS_NAME=supos
VOLUMES_PATH=/c/Users/Free/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work, )
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192
ENTRANCE_PORT=8

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US

# Operating system version
OS_VERSION=V1.00.00.05-C

# MQTT TCP port
OS_MQTT_TCP_PORT=1883

# MQTT WebSocket TLS port
OS_MQTT_WEBSOCKET_TLS_PORT=8084

# Login path for the system
OS_LOGIN_PATH=/supos-login

# Enable or disable authentication (true/false)
OS_AUTH_ENABLE=true

# Type of large language model used
OS_LLM_TYPE=ollama

# Model and Key for oepnai (Enter your api key, can be empty)
OPENAI_API_MODEL=gpt-4o
OPENAI_API_KEY=

# Below are the environment variables required for using the built-in supos MCP with the MCP Client (if applicable, can be empty)
# Langsmith API key
LANGSMITH_API_KEY=
# Agent service address (default: http://localhost:8123)
AGENT_DEPLOYMENT_URL=http://mcpclient:8123
# open API key for supos, used for external access to the system
SUPOS_API_KEY=4174348a-9222-4e81-b33e-5d72d2fd7f1e
# The address of the MQTT server of supos that needs to be accessed
SUPOS_MQTT_URL=tcp://emqx:1883/mqtt

#Variables from older versions, will be deleted for next iteration, please do not modify
MULTIPLE_TOPIC=false

#-----OAUTH CONFIG-----
OAUTH_REALM=supos
.env [unix] (10:22 20/05/2025)
".env" [unix] 69L, 1988B

```

4. 保存文件并开始安装 Tier0:

```

:wq!
bash bin/install.sh

```

5. 选择默认配置, 等待安装完成。

```
MINGW64/d/supOS-CE
Free@DESKTOP-JK7K19R MINGW64 /d/supOS-CE (main)
$ bash bin/startup.sh
Choose VOLUMES_PATH: (Press Enter for default: [/c/Users/Free/volumes/supos/data])
Choose IP address for ENTRANCE_DOMAIN (Press Enter for default: [192.168.1.1]):
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /d/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
golioth-websocket-datasource/plugin.json
golioth-websocket-datasource/README.md
Info: success to create folder: /c/Users/Free/volumes/supos/data
time="2025-05-20T10:22:57+08:00" level=warning msg="D:\\supOS-CE\\docker-compose-8c16g.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 14/14
  ✓ Network supos_default_network Created 0.0s
  ✓ Container postgresql Healthy 62.0s
  ✓ Container eventflow Started 0.8s
  ✓ Container chat2db Started 0.8s
  ✓ Container konga Started 0.4s
  ✓ Container emqx Started 1.0s
  ✓ Container portainer Started 0.6s
  ✓ Container frontend Started 0.6s
  ✓ Container hasura Started 61.3s
  ✓ Container nodered Started 1.2s
  ✓ Container keycloak Healthy 81.9s
  ✓ Container grafana Started 61.4s
  ✓ Container backend Started 82.1s
  ✓ Container kong Started 82.3s
>> start to init nodered modules ...
added 55 packages in 4s
28 packages are looking for funding
  run 'npm fund' for details
npm warn deprecated node-opcua-client-crawler@2.127.1: true
```

6. 在浏览器中访问 Tier0 并登录。

卸载 Tier0

1. 进入路径 Tier0-Edge/deploy/bin。
2. 执行脚本 `uninstall.sh`:

```
bash uninstall.sh
```

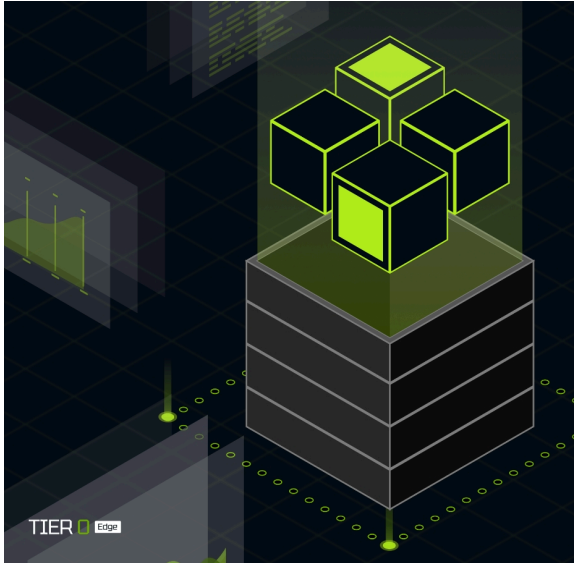
① 信息

- `clean-all.sh` 会清除所有项目数据并卸载Tier0。
- 镜像不会被删除。
- 如需重新安装 Tier0，请确保拉取所有最新版本的镜像（基于镜像标签）。

登录

1. 在浏览器中输入已设置的域名和端口，访问平台。

2. 在登录页面，输入用户名 `tier0` 和密码 `tier0`。



登录到您的账户

用户名

密码

登录

3. 单击登录。

操作指南

建立数据模型

Tier0 采用统一命名空间（UNS）将数据组织成清晰的树形结构，模型中的每个节点都会按照树形图自动生成 MQTT 主题，方便获取实时数据。

补充参数说明

- 文件夹-更多设置

项目	说明
扩展属性	为文件夹添加扩展属性（例如：单位）。
模板	选择模板并继承其属性，详见 创建模板 。
生成模板	单击  为文件夹添加属性，同时可选择生成相同模板供后续使用。

- 文件-文件类型

类型	说明
状态	作为 API 接收系统状态数据。
操作	作为 API 执行增删改查（CRUD）命令。
指标	实时数据（例如：设备温度）。

- 文件-属性生成方式

属性生成方式	说明
自定义	自定义属性。
模板	从选中的模板继承属性。
自动解析	通过 JSON 文本生成相同结构的属性。

- 文件-可选行为

行为	说明
启用历史数据	为选中文件开启历史数据存储功能。

如何创建数据模型

基于简单的文件夹-文件结构，可将数据层级定义为树形视图。

手动创建模型

① 备注

factory/equipment/CNC 将作为示例，其中 `factory` 和 `equipment` 为文件夹，`CNC` 为文件。

1. 登录 Tier0，即进入数据建模。
2. 在工厂模型栏，单击  > 新建文件夹（例如：`factory`）。

新建文件夹

命名空间 ②

* 名称

▼ 更多设置

* 别名 ②

__02a670bbfed44188ba3d

显示名称

文件夹描述

扩展属性

⊕

模板

自定义

生成模板

☒

键 ②

名称

类型

▼

长度

显示名称(...)

备注(选填)

⊖

⊕

保存

- 3. 根据需求填写文件夹名称，别名等信息，单击保存。
- 4. 选中equipment，单击  > 新建文件，在其下添加文件（例如：CNC）。

新建文件

命名空间 ?

equipment/

* 名称

文件类型 ?

☒ 状态

☐ 操作

☐ 指标

属性生成方式 ?

☒ 自定义

☐ 模板

☐ 自动解析

键 ?

名称

类型

长度

显示名称...

备注(选填)

更多设置

* 别名 ?

__f0cb57cfb5b24622bb30

显示名称

文件描述

标签 ?

扩展属性

可选行为

☐ 启用历史数据 ?

保存

- 5. 输入文件相关信息，单击保存。

导入模型

- 1. 登录 Tier0，即进入数据建模。
- 2. 单击右上角的导入。



3. 导入 JSON 创建模型：

- 切换到UNS栏，直接输入 JSON 文本。

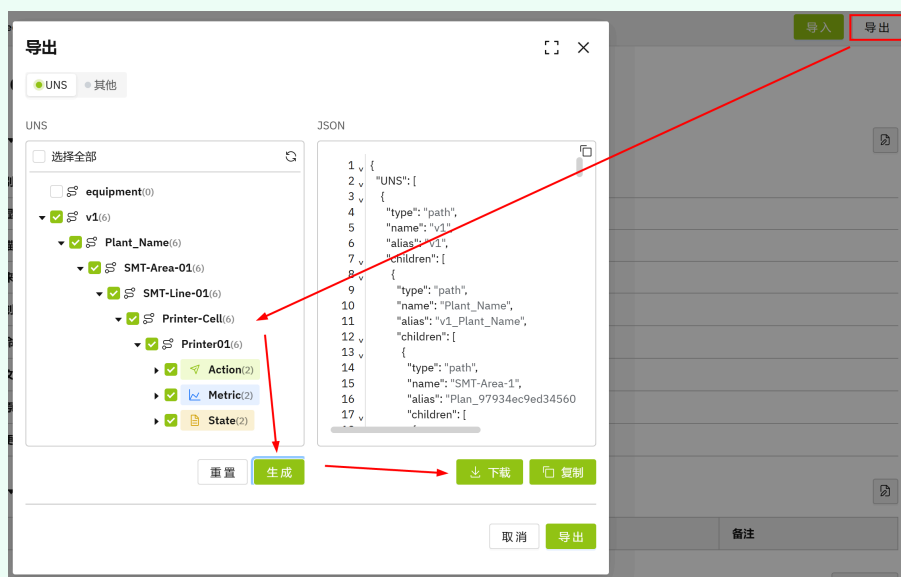


- 在UNS栏，单击下载模板，按模板规则填写模型内容后切换到全部栏上传。

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION",
  "Label": [{"name": "debug"}, {"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
                              "topicType": "STATE"
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

✓ 提示

可手动添加文件夹和文件后导出，将其作为导入示例参考。



4. 单击保存。

可选功能

创建模板

1. 在数据建模页面单击模板栏。
2. 单击 ，输入模板信息。

✓ 提示

单击来源可选择已添加的模型并继承其属性。

新建模板 ×

* 名称

模板描述

键 ↔ 来源

名称	类型	长度	显示名称(...)	备注(选填)	
+					

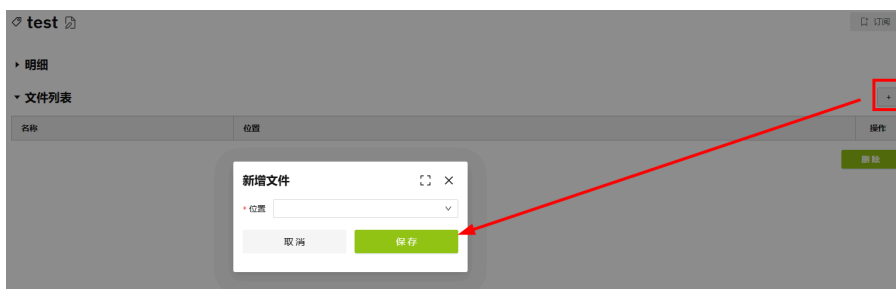
保存

3. 单击保存。

创建标签

标签用于对数据模型进行分类管理。

1. 在数据建模页面单击标签栏。
2. 单击 ，输入标签名称。
3. 单击保存，再单击  为标签添加文件。



4. 单击保存。

导出数据

Tier0 支持导出UNS模型以及其他数据，如数据连接，事件流程及数据看板。

1. 在数据建模页面单击右上角导出。

2. 单击其他栏，选择对应的数据类型。

导出

● UNS

● 其他

数据连接

请选择

事件流程

请选择

数据看板

请选择

取消

导出

3. 单击导出。

连接数据源

如何添加数据源

在数据建模下完成数据模型构建后，需为模型连接真实数据，以便后续处理和使用。

手动添加数据流

1. 登录平台，进入数据管理 > 数据连接。

数据连接

请输入名称或描述进行搜索

搜索

新建数据连接

名称	连接模板	描述	创建时间	创建人	操作
Equipment/Metric/CNC 运行中	node-red		2025/12/02 14:21:13	supos	
Equipment/状态/CNC 运行中	node-red		2025/12/02 14:12:01	supos	

总共 2 项 < 1 > 18 条/页

总共 2 项 < 1 > 18 条/页

序号	项目	说明
1	搜索	搜索现有数据流。
	新建数据连接	添加一条新的数据流。
2	数据流列表	显示数据流信息，单击流名称可进入数据流编辑页面。
3	显示方式	切换数据流的卡片式/列表式显示。
4	流操作	对数据流进行复制/编辑/删除操作。

- 单击**新建数据连接**，输入名称后单击保存。
- 单击刚创建的数据流，根据数据源类型拖拽对应节点（例如 **modbus** 节点）至画布。
- 拖拽一个 **MQTT输出** 节点，并完成配置：

- 添加内置代理：**emqx:1883**。

Edit mqtt out node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server Port

☒ Connect automatically

☐ Use TLS

Protocol

Client ID

Keep Alive

Session ☒ Use clean session

- 设置订阅主题为 **数据建模** 中的数据模型路径。

例如：数据模型结构为 **factory/equipment/cnc**，则主题设置为 **factory/equipment/cnc**。

Edit mqtt out node

Delete Cancel Done

Properties

Server emqx:1883 ▼ ✎ +

Topic factory/equipment/cnc

QoS ▼ Retain ▼

Name Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

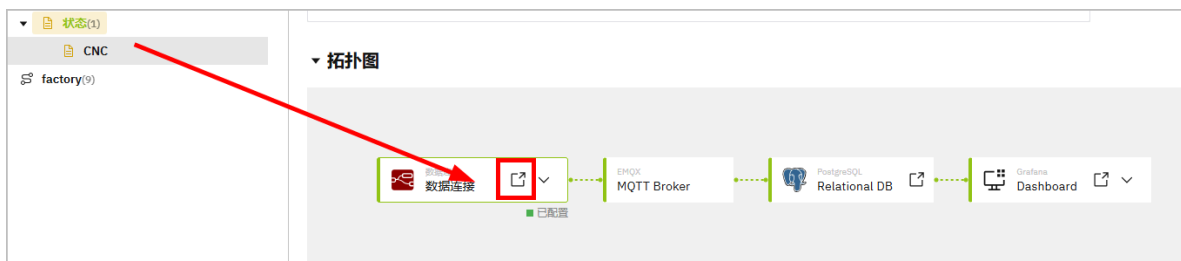
5. 单击右上角部署，触发数据流后，数据将发送至数据建模中对应的模型。

通过生成的流配置数据源

① 备注

创建文件时选择模拟数据，系统才会自动生成数据流。

1. 登录平台，进入数据管理 > 数据建模，选择目标文件。
2. 向下滚动至拓扑图区域，单击数据连接右侧的图标，跳转至数据连接下自动生成的数据流页面。



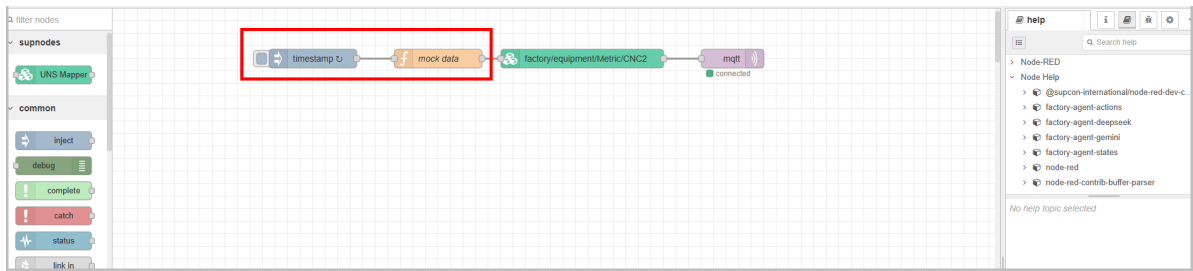
✓ 提示

单击 ▼ 改变绑定的数据流。

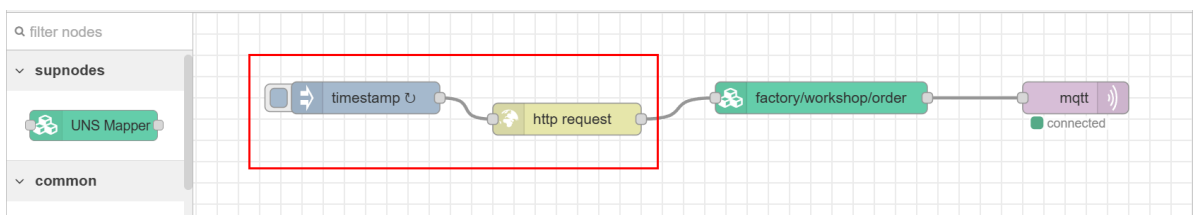
3. 修改数据源。

① 信息

确保源节点返回的数据格式为 JSON对象，目前平台仅支持解析该数据类型。



4. 删除模拟数据的函数节点，将源节点与后续流程连接。



5. 单击右上角部署，然后触发数据流。

6. 返回数据建模，检查数据是否已回传至该文件。

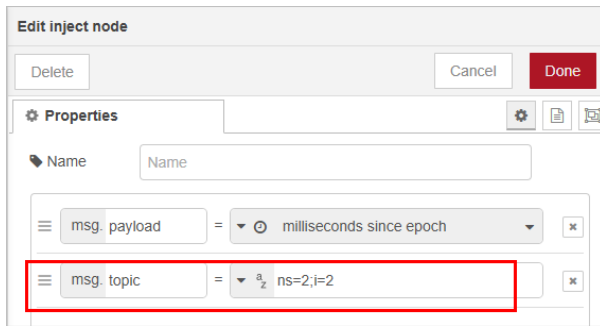
消息体		
键	值	更新时间
actualRuntime	1285	11:24:55 16/05/2025
plannedRuntime	1440	11:24:55 16/05/2025

常规数据源

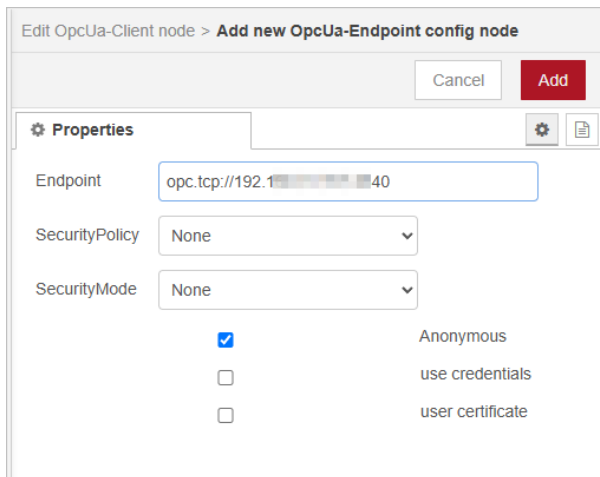
连接 OPC UA

从OPCUA服务器读取数据

1. 在 数据连接 中，拖拽一个 `inject` 节点。
2. 双击该节点，输入 OPC UA 服务器中的 节点ID。



3. 拖拽一个 **OpcUa-Client** 节点，然后双击该节点。
4. 单击 Endpoint 右侧的 **+** 图标，输入你的 OPC UA 服务器信息。



5. 单击 Add，将 Action 设置为 READ。
6. 其他参数保持默认配置，单击 Done。

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint:

Action:

Certificate:

Local certificate file, fixed path:

Local private key file, fixed path:

Use transport settings: ☐

Max ChunkCount:

Max MessageSize:

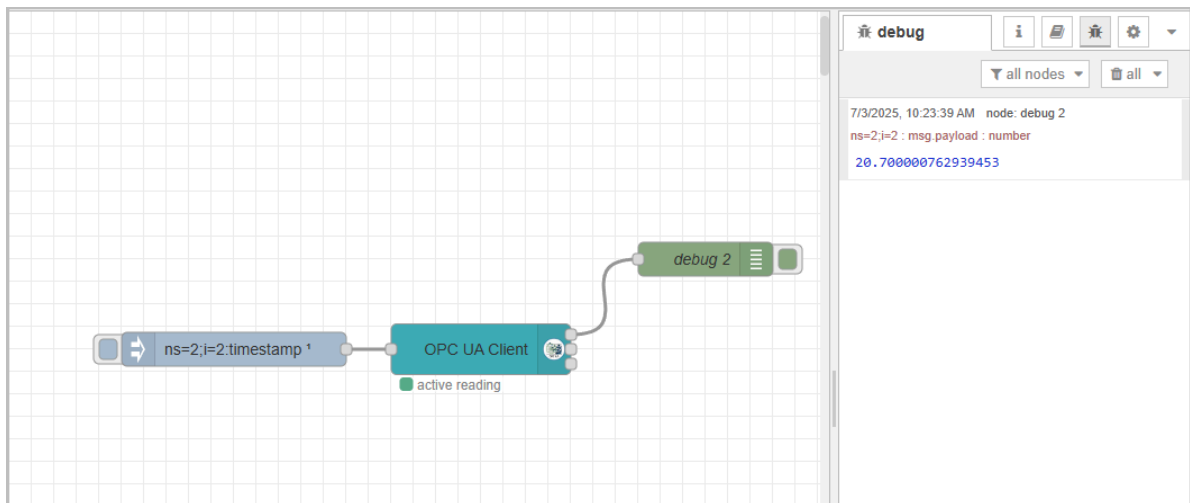
Receive BufferSize:

Send BufferSize:

Set statuscode and timestamp: ☒

Keep session alive: ☒

7. 将 **inject** 节点与 **OpcUa-Client** 节点连接，最后在末端连接一个 **debug** 节点。
8. 单击右上角 **Deploy**，然后单击 **inject** 节点触发流程。

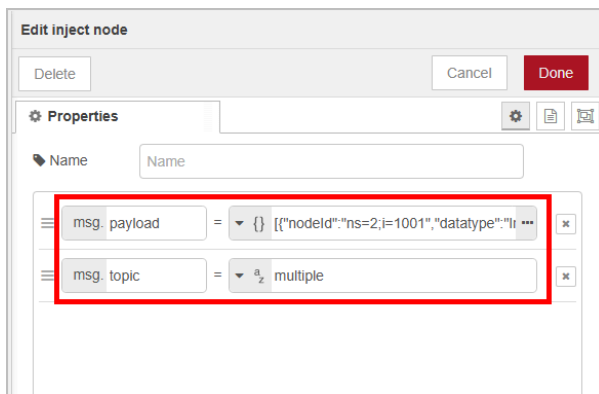


从OPCUA服务器订阅数据

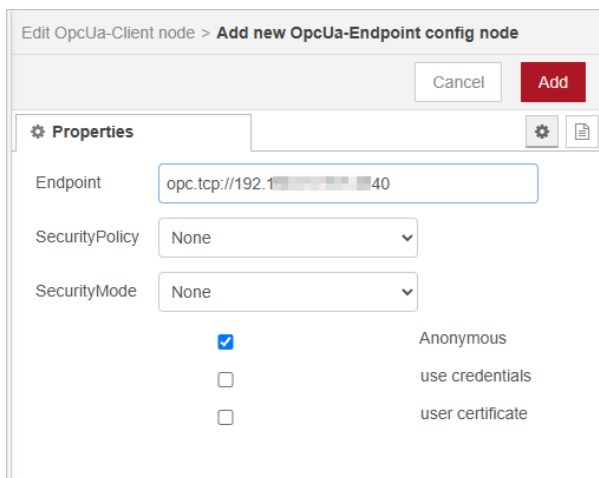
1. 在 **数据连接** 中，拖拽一个 **inject** 节点。
 2. 双击该节点，输入 OPC UA 服务器中的 **节点ID**。
- msg.payload:

```
[
  {
    "nodeId": "ns=2;i=1001",
    "datatype": "Int32"
  },
  {
    "nodeId": "ns=2;i=1002",
    "datatype": "Int32"
  }
]
```

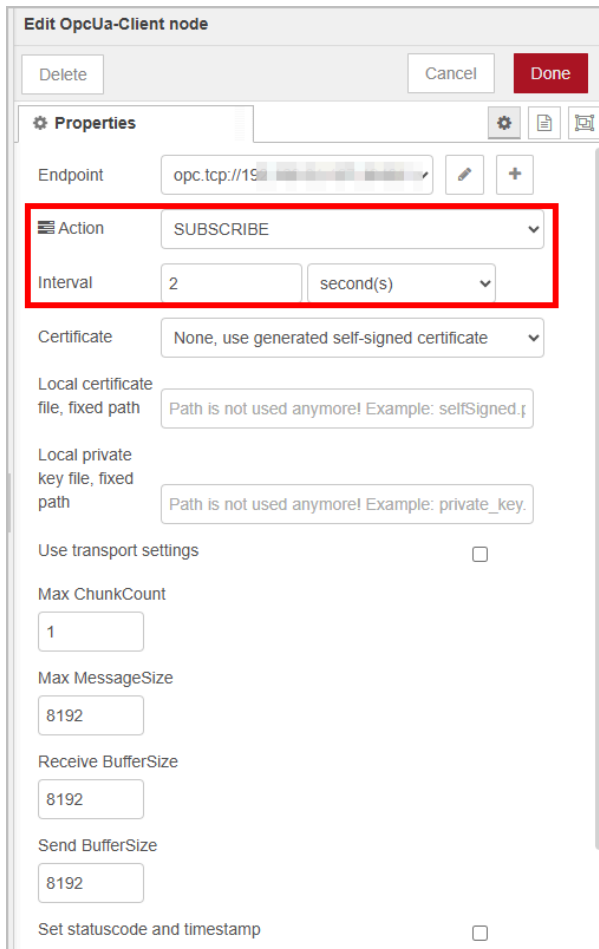
- msg.topic: multiple



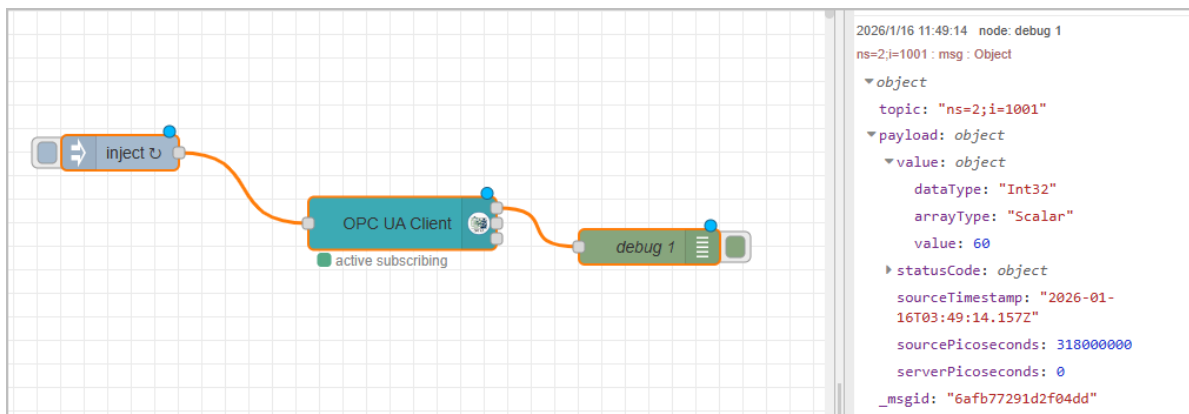
3. 拖拽一个 **OpcUa-Client** 节点，然后双击该节点。
4. 单击 Endpoint 右侧的  图标，输入你的 OPC UA 服务器信息。



5. 单击 Add，将 Action 设置为 SUBSCRIBE。
6. 设置订阅间隔，其他参数保持默认配置，单击 Done。



7. 将 **inject** 节点与 **OpcUa-Client** 节点连接，最后在末端连接一个 **debug** 节点。
8. 单击右上角 **Deploy**，然后单击 **inject** 节点触发流程。



连接 OPC DA

1. 在 **数据连接** 中，拖拽一个 **inject** 节点。
2. 拖拽一个 **opcda-read** 节点，然后双击该节点。
3. 单击 **服务器** 右侧的 **+**，输入你的 OPC DA 服务器信息。

Edit opcda-read node > Add new opcda-server config node

Cancel Add

Properties

Name: opcda

Address: 192.168.31.142

Domain: "

User Name: admin

Password:

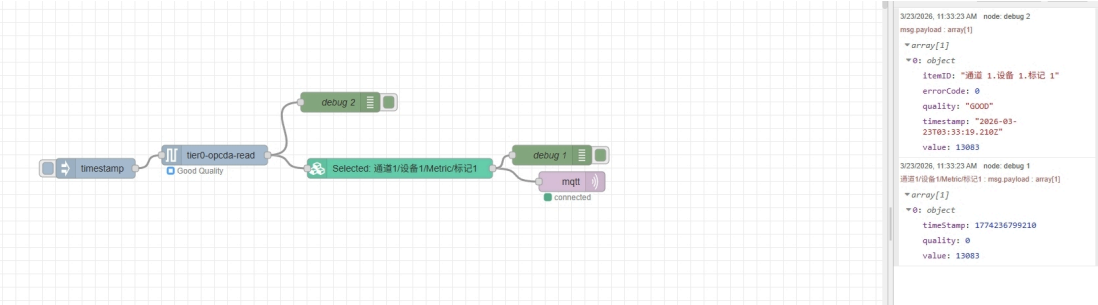
Clsid: 7BC0CC8E-482C-47CA-ABDC-0FE7F9C6E729

Timeout: 5000 ms

Browse Export

参数	说明
地址 (Address)	OPC DA 服务器的 IP 地址。
域名 (Domain)	OPC DA 服务器的域名。
用户名/密码 (Username/Password)	登录 DA 服务器所用的用户名和密码。
应用标识 (Clsid)	所使用的 DA 连接服务的应用 ID（例如 KepServer）。

- 单击 **Browse** 查看已连接的通道。
- 单击 **Add**，在 **Items** 下添加所需的项目。
- 将 **inject** 节点与 **opcda-read** 节点连接，最后在末端连接一个 **debug** 节点。
- 单击右上角 **Deploy**，然后单击 **inject** 节点触发流程。



连接 Modbus

- 在 **数据连接** 中，拖拽一个 **Modbus-Read** 节点。

2. 双击该节点，单击 Server 右侧的 ，输入你的 Modbus 服务器信息。

Edit Modbus-Read node > Add new modbus-client config node

Cancel Add

Properties

Settings Queues Optionals

Name modbus

Type TCP

Host 192.168.1.100

Port 502

TCP Type DEFAULT

Unit-Id 1

Timeout (ms) 1000

Reconnect on timeout ☒

Reconnect timeout (ms) 2000

3. 单击 Add，输入待连接数据的相关信息。

Edit Modbus-Read node

Delete Cancel Done

Properties

Settings Optionals

Name modbus

Topic Topic

Unit-Id

FC FC 3: Read Holding Registers

Address 0

Quantity 1

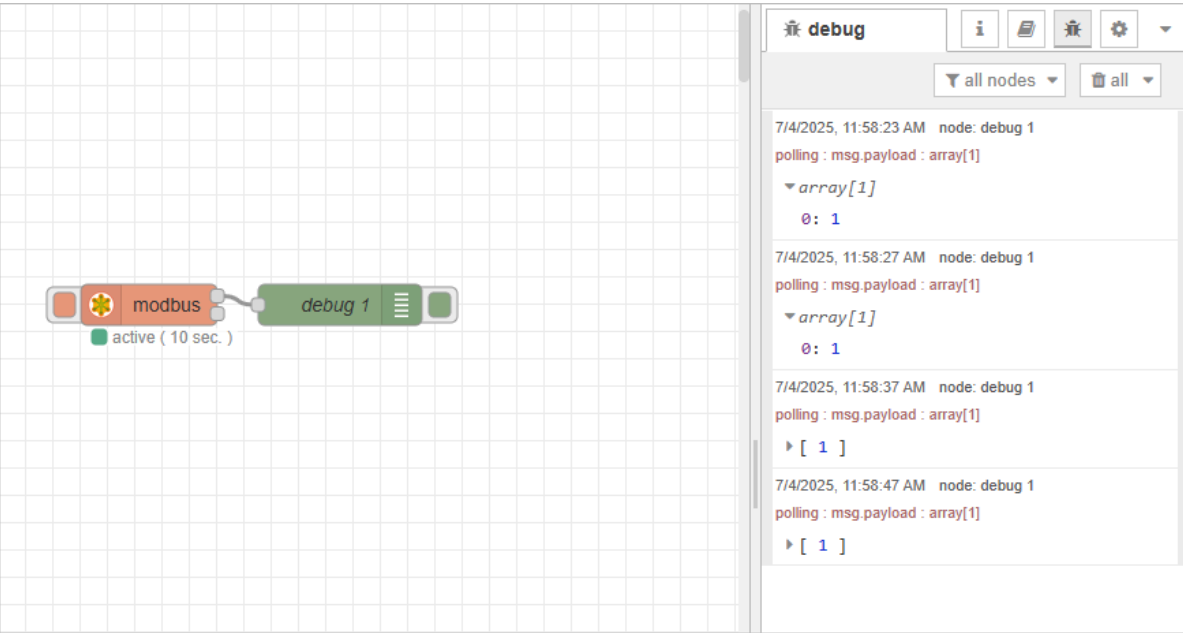
Poll Rate 10 second(s)

Delay to activate input ☐

Server modbus

参数	说明
名称 (Name)	节点的显示名称（仅用于可视化识别）。
主题 (Topic)	可选参数，将传递至输出消息的主题字符串。
从站地址 (Unit-Id)	目标设备的 Modbus 从站 ID（通常为 1 - 247）。
功能码 (FC)	指定 Modbus 操作的功能码（例如：功能码 3 = 读取保持寄存器）。
起始地址 (Address)	读取的起始寄存器地址（通常为零基地址）。
读取数量 (Quantity)	连续读取的寄存器个数。
轮询频率 (Poll Rate)	节点轮询 Modbus 设备的间隔时间（例如：每 10 秒一次）。
输入激活延迟 (Delay to activate input)	可选参数，轮询输入信号激活前的延迟时间。
服务器 (Server)	已配置的 Modbus 服务器引用（包含 IP、端口、协议等信息）。

- 4. 单击 Done。
- 5. 在末端连接一个 debug 节点。
- 6. 单击右上角 Deploy，然后单击 Modbus-Read 节点触发流程。



连接 MQTT

1. 在 数据连接 中，拖拽一个 `mqtt in` 节点。
2. 双击该节点，单击 Server 右侧的  。

Edit mqtt in node > Add new mqtt-broker config node

Cancel

Add

Properties

Name

Name

Connection

Security

Messages

Server

e.g. localhost

Port

1883

☒ Connect automatically

☐ Use TLS

Protocol

MQTT V3.1.1

Client ID

Leave blank for auto generated

Keep Alive

60

Session

☒ Use clean session

参数 (Parameter)	说明 (Description)
名称 (Name)	MQTT 代理配置的可选名称（仅用于内部参考）。
服务器 (Server)	MQTT 代理的主机名或 IP 地址（例如：localhost 或 broker.hivemq.com）。
端口 (Port)	用于连接 MQTT 代理的端口（非 TLS 连接默认值为 1883，TLS 连接默认值为 8883）。
自动连接 (Connect automatically)	启用后，Node-RED 会在流程启动时自动尝试连接代理。
使用 TLS (Use TLS)	若代理要求安全的 TLS/SSL 连接，需启用此选项。
协议 (Protocol)	用于连接的 MQTT 协议版本（例如：MQTT v3.1.1 或 v5.0）。
客户端 ID (Client ID)	可选的客户端 ID。若留空，系统会自动生成唯一 ID，且每个客户端的 ID 必须唯一。
保活时间 (Keep Alive)	与代理通信的最大间隔时间（单位：秒），用于检测断开的连接。
使用干净会话 (Use clean session)	启用后，不会与代理保持持久会话；未启用时，会话状态（如订阅信息）会被保留。
用户名 (Username)（安全标签页）	若需要，填写用于 MQTT 代理身份验证的用户名。
密码 (Password)（安全标签页）	与用户名对应的代理身份验证密码，会被安全存储。

3. 单击 Add，输入需要订阅的主题。

4. 单击 Done。

Edit mqtt in node

Delete Cancel Done

Properties

Server emqx

Action Subscribe to single topic

Topic factory/RCS/AGV

QoS 2

Output auto-detect (parsed JSON object, string or buff)

Name Name

5. 在末端连接一个 `debug` 节点。
6. 单击右上角 部署 (Deploy)，当该主题接收到数据时，流程会被触发。

debug

all nodes all

7/4/2025, 2:24:45 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

object

no: 66
position: "3fcLKwHDAvthuZLEwW8V"
state: "DqGG213IuzMDT73GnVS3"
battery: "62.91"

7/4/2025, 2:24:55 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

{ no: 60, position: "6CcOvOZfJM8zaCCJA15e", state: "zmP15enJhhK064mK16wV", battery: "63.99" }

factory/RCS/AGV debug 1

connected

连接文件

① 信息

在数据连接中处理文件前，需将文件放入对应的容器中。

```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/  
//将文件从本地复制到服务器  
docker cp /root/config.json <容器名称或ID>:/app/config.json  
//将文件从服务器复制到容器
```

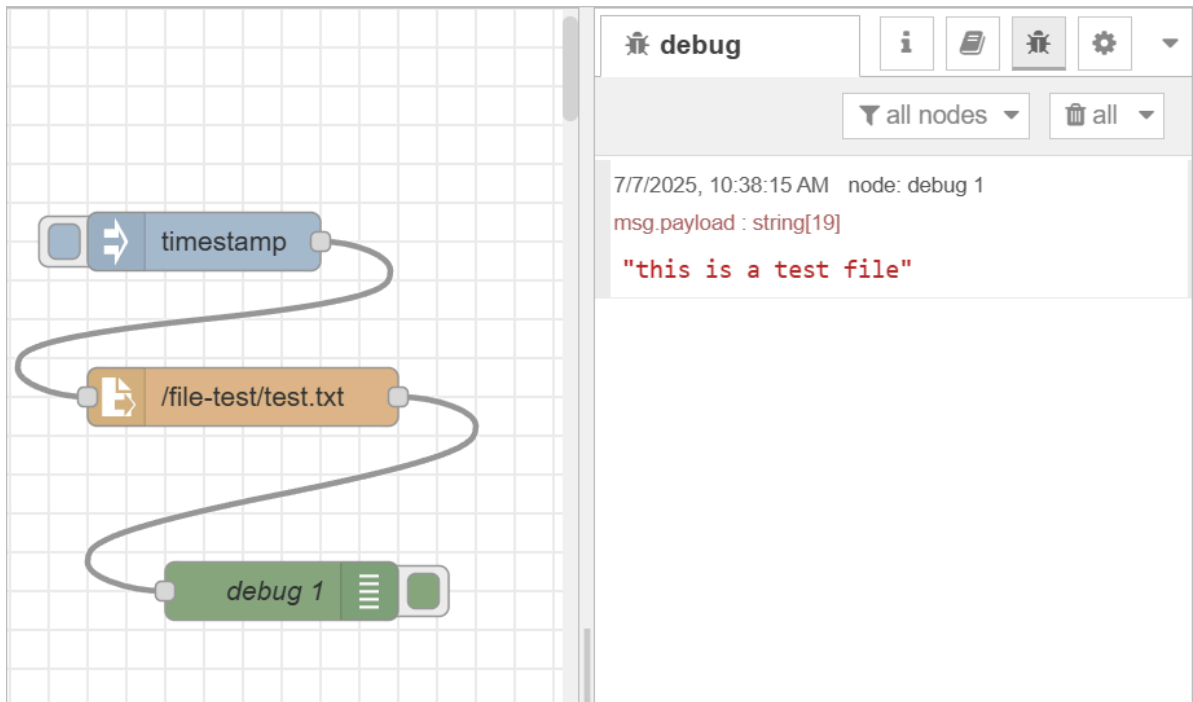
✓ 常用Docker命令

命令	说明
<code>docker ps</code>	列出所有正在运行的Docker容器。
<code>docker ps grep [container]</code>	根据指定的容器信息筛选正在运行的容器。
<code>docker exec -it [container] /bin/sh</code>	通过 <code>/bin/sh</code> 在运行中的容器内启动一个 shell会话。
<code>mkdir [directory]</code>	在容器内创建一个指定名称的新目录。

连接文本文件

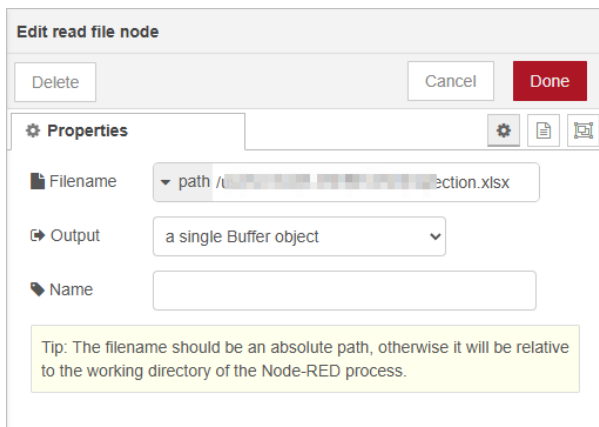
1. 在 数据连接 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `read file` 节点，然后双击该节点。
3. 输入文件路径和输出配置。

4. 单击 完成，然后在末端连接一个 `debug` 节点。
5. 单击右上角 部署，接着单击 `inject` 节点触发流程。

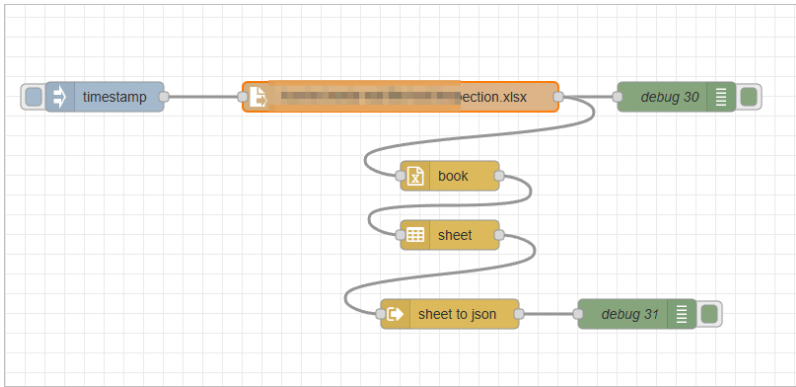


连接Excel文件

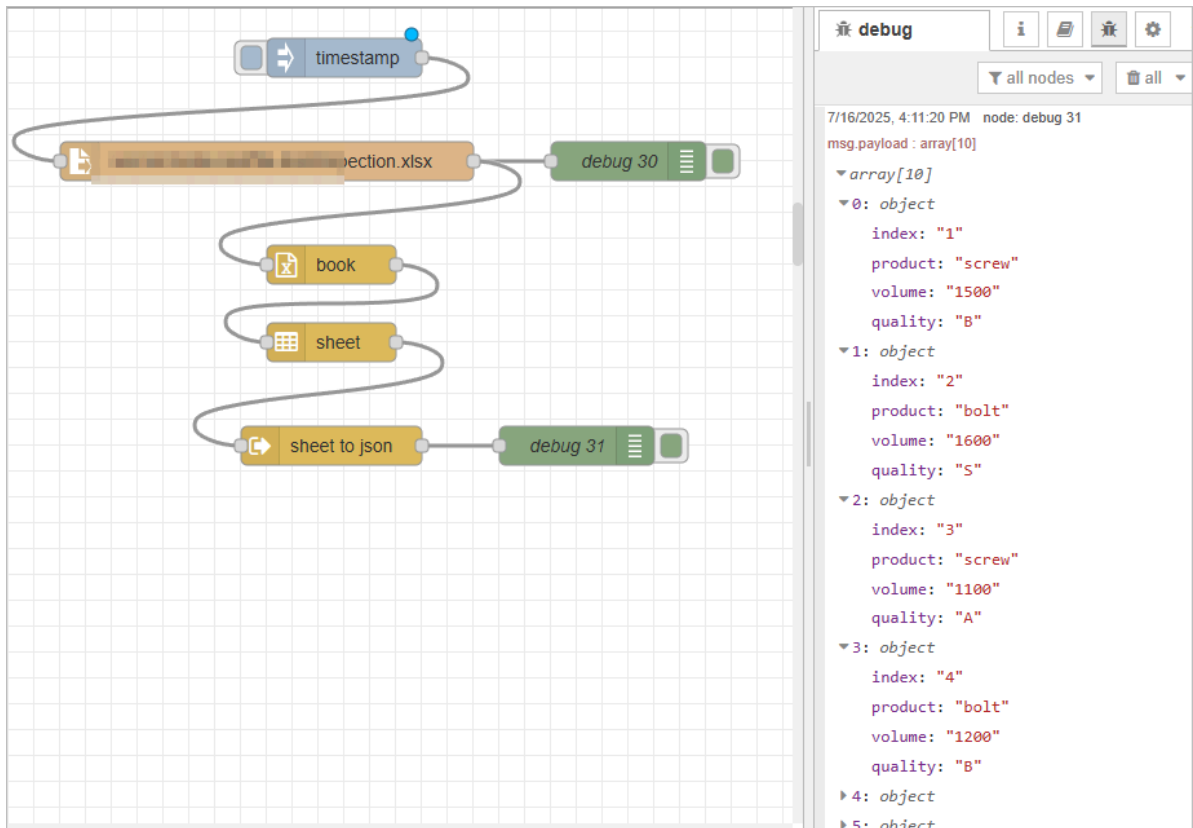
1. 在 数据连接 中，拖拽一个 **inject** 节点。
2. 拖拽一个 **read file** 节点，然后双击该节点。
3. 输入文件路径和输出配置。



4. 单击 **Done**，然后按顺序连接 **book**、**sheet** 和 **sheet to json** 节点，将表格数据转换为JSON格式。



5. 单击右上角 Deploy，接着单击 **inject** 节点触发流程。



连接 RestAPI

1. 在 数据连接 中，拖拽一个 **inject** 节点。
2. 拖拽一个 **http request** 节点，然后双击该节点。
3. 输入API相关信息。

✓ 提示

需将 Return 设置为 a parsed JSON object，以便平台的数据建模模块解析。

Edit http request node

Delete Cancel Done

Properties

Method GET

URL http://

Payload Ignore

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a parsed JSON object

Tip: If the JSON parse fails the fetched string is returned as-is.

Headers

+ add

5. 单击 完成，然后在末端连接一个 **debug** 节点。
6. 单击右上角 **Deploy**，接着单击 **inject** 节点触发流程。

